Handoff 43: The learner's own files (master run, ENG-08b, decision 30)

Date: 2026-11-09 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#51), not merged, not deployed. **No migration, no new dependency, no real lesson uses it yet.** Closes a way round the import guard that exists on `main` today (section 5).

> **How to use this file:** upload it with `docs/plan/browser-checks.md` (sections 9, 10 and 12) and handoffs 34 and 35. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing ENG-08b for Python Academy: a learner can have up to 3 files of their
own, in
editor tabs, which their program imports. Decision 30 says a file named random.py or os.py
must not
shadow or get past the import guard, and the guard must apply inside learner files. Check
that the
worker meets that, that the learner's files never reach the API, and that nothing changes
for
lessons without learner files. Say whether "tell a learner's module by where it lives" is
sound.
```

Run plan details

Field	Value
Stage	ENG-08b (backlog), before unit 7. Owner decision 30: its own stage
Base	`main`, at `34527cb`, after #40 (ENG-08) merged; merged again after #46 and #50 merged
Part 2 boxes relied on	**None**
Merge order	#46 and #50 are merged, and this branch is updated. No other PR needs to merge first

1. Step 0

`git fetch --all` and `gh pr list --state all`: #46 and #50 open, #40 merged. Worktree `../python-academy-modules`; the main folder was not switched.

2. Handoff record

Field	Value
Task	A learner's own second (and third, and fourth) file, for `py.own-modules`
Human goal	Unit 7 can teach "write and import your own module", without weakening the import guard
Non-goals	The `py.own-modules` lesson itself; saving files; folders of learner files; packages
Data and compatibility	None. New optional content field `modules`; lesson views gain `exampleModules` and `challengeModules` only when set
Deployment status	Not requested

Files changed

File	Change
`backend/app/content.py`	`modules` on examples and challenges; name rules; given files may not be named like a Python module either
`backend/tests/case_runner.py`	The test runner writes the learner's files and forgets them between cases, as the worker does
`backend/tests/test_learner_modules.py`, `backend/tests/fixtures/learner_modules/`	New tests and a TEST FIXTURE lesson
`frontend/public/python-worker.js`	Allowed modules load first; the program's folder goes last on the import path; the guard applies inside learner files; learner files are forgotten between cases
`frontend/public/check-runner.js`	Passes the learner's files to the worker
`frontend/app/adventure.tsx`, `style.css`	File tabs above the editor, when a card has learner files
`frontend/tests/learner-modules.spec.ts`, `fixtures/learner-modules-api.json`	New browser tests and the recorded API answers
`docs/plan/browser-checks.md`	Section 12, and updates to sections 9 and 10

3. Security note

The import guard is a guide, not a sandbox, and that is unchanged (section 9 of `browser-checks.md`). The boundary is the worker: no network, its Content-Security-Policy, nothing from the page. **The learner's files never leave the browser:** a browser test checks the request body is exactly the recorded one, with no file in it. A remaining, documented way round the guard: `__import__("os", vars(random))` from a learner's file, like `random._os`. I confirmed it still works.

4. Decisions and why

Decision	Options considered	Reason	Who decided
A learner's module is any module whose file is under `/home/pyodide`	A list of names; a flag set on import	Where a module lives can't be faked by naming it, and it also covers a file the program writes while it runs	Implementer
Load every allowed module before the learner's code, **and** put the program's folder last on the path	Either one alone	Pre-loading covers the allowed modules. The path order covers modules Python loads later, such as `_strptime`. A deliberate break showed one is not enough (section 6)	Implementer
Refuse names Python already has, in content **and** in the worker	Content only	The worker check holds even if content validation is ever skipped	Implementer, following decision 30
Up to 3 learner files, top level only	Any number, in folders	Enough for `py.own-modules`; easier to explain	Implementer (**assumption 1**)
The main file's tab is "main.py"	"story.py", as the editor header says elsewhere	It's the name learners meet in real projects	Implementer (**assumption 2**)
Errors inside a learner's file point to the program's line that called it	Report the file and its line	The judge and the page know only lines of the program. Section 7	Implementer
Learner files never go to the API	Send them, for construct rules	Learner code leaves the browser only as the reports the ENG-03 contract defines	Plan rule

5. Bugs found and how they were fixed

A way round the guard on `main`: a program could write `datetime.py` while running, import it, and from inside it import `os`. The guard trusted any module's own code, wherever its file was, and the program's folder came before Python's library on the import path. Found by running it in Pyodide before writing any code. Fixed by all three of: allowed modules loaded first, the folder last on the path, and the guard no longer trusting files under `/home/pyodide`.

A given file could also be named like a module. Content now refuses a top-level given file such as `random.py`. `data.py`, or `notes/random.py` in a folder, is still fine.

6. How it was tested

Command	Result
Backend, SQLite: `DATABASE_URL=sqlite:///<temp>.db python -m pytest -q`	**871 passed, 11 skipped**
`npx tsc --noEmit` and `npm run build`	OK
`CI=1 npx playwright test tests/learner-modules.spec.ts`	**18 passed**
Related earlier specs: allowed imports, files, browser checks, function checks, hidden calls, keyboard, lesson units, unit 1 snapshot	**94 passed**
Backend on PostgreSQL, and the full browser suite	**In CI on this PR.** The full browser suite has not been reliable on this Mac (handoff 35 section 8)

Mocks: page tests use the recorded real API answers, matched exactly. Python, the worker and the page are real.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. The guard trusts learner files again	**1**
B2. The program's folder first on the path	**0 at first.** Pre-loading hid it. A new test, a late-loaded `_strptime.py`, now fails: **1**
B3. Learner modules not forgotten between cases	**1**
B4. The worker doesn't check names	**1**
B5. The page sends the learner's file to the API	**3**
B6. Learner files can't be imported	**7**

7. Known limitations

- **No real lesson uses this yet.** `py.own-modules` is unit 7.
- **An error inside a learner's file is reported at the program's line that called it,** not at its own line. The explanation is about the right error, and Python's full message, under "Show Python's message",

names the file and line.
- `AttributeError` (for example, `total` not yet written in `scores.py`) has no friendly explanation of its own today, so it gets the general one, and a hint can't be tagged with it.
- **Construct rules see only the program.**
- **Drafts of learner files** live in the page's memory, like the program's drafts, and are lost on reload.
- **Not tested on Safari or Firefox, or with a screen reader.** The tabs follow the WAI-ARIA tabs pattern and have keyboard tests.

8. Merging with #46 and #50

Both merged first, on 2026-11-09, so I merged `main` into this branch. Five files conflicted, all lines both sides changed, and I kept both: the worker gets each case's reference (ENG-10) and the learner's files; the editor keeps #50's instruction about Esc; `browser-checks.md` has section 11 (ENG-10, now marked built) and then section 12. After the merge: backend **894 passed, 11 skipped** on SQLite; types and build OK; **89 passed** in the learner-files, reference, keyboard, accessibility, file, unit 1 and allowed-imports specs. The full browser suite runs in CI.

9. Live checks for you

None until a lesson uses learner files. After deploying, check that a unit 2 or 3 lesson works as before, with no CSP reports in DevTools → Console.

10. Questions for you

1. **Is "main.py" the right name for the program's tab?** *Recommended: yes.*
2. **Should errors inside a learner's file point to that file's own line?** *Recommended: yes, in the stage that builds `py.own-modules`. It needs the judge to know about files.*

11. For the docs sync

Where	What
Backlog	ENG-08b `in_review` (#51, handoff 43). ENG-08 `done` (#40)
Stage table	"43. Learner files | ENG-08b | in_review (#51, handoff 43)"
Decision log	Decision 30 met; the way round the guard found and closed

12. Next concrete action

Merge this PR once CI is green.

13. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-modules status --short   # expect nothing
git worktree remove ../python-academy-modules
git worktree prune
git branch -d eng/learner-modules
```